

SLIDE 1 — What is Combining Loops with Arrays?

Definition:

A loop with an array is used to process multiple array elements automatically.

Simple idea

Array → Loop → Visit Each Element

Example:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]);  
}
```

Explanation:

The loop visits each index one by one.

i = 0 → 10

i = 1 → 20

i = 2 → 30

i = 3 → 40

i = 4 → 50

Output:

10 20 30 40 50

 Remember: Loop + Array = Process many elements easily!

SLIDE 2 — How Does the Loop Access an Array?

Structure

```
for (int i = 0; i < size; i++) {  
    array[i];  
}
```

Example

```
int marks[5] = {80, 90, 75, 85, 95};
```

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", marks[i]);  
}
```

Explanation:

i = 0 → marks[0]

i = 1 → marks[1]

i = 2 → marks[2]

i = 3 → marks[3]

i = 4 → marks[4]

★ Remember

● i changes → index changes

● Loop stops when i < size becomes FALSE

SLIDE 3 — Taking Array Input Using a Loop

A loop can be used to **enter values into an array**.

Example

```
int numbers[5];
```

```
for (int i = 0; i < 5; i++) {  
    scanf("%d", &numbers[i]);  
}
```

Explanation:

The loop takes input for each position.

Input → numbers[0]

Input → numbers[1]

Input → numbers[2]

Input → numbers[3]

Input → numbers[4]

Example Input:

10 20 30 40 50

Output stored:

[10] [20] [30] [40] [50]

 **Remember:** Loop makes entering many values easy!

SLIDE 4 — Processing Array Elements

Loops can perform calculations on array elements.

Example: Find Sum

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
int sum = 0;
```

```
for (int i = 0; i < 5; i++) {  
    sum = sum + numbers[i];  
}
```

Explanation:

sum = 0

↓

+10 → 10

↓

+20 → 30

↓

+30 → 60

↓

+40 → 100

↓

+50 → 150

Output:

Sum = 150

Basic Pattern

START → LOOP → ACCESS → PROCESS → REPEAT

SLIDE 5 — RECAP

Loop + Array = Powerful Combination! 

Flow

ARRAY



LOOP STARTS



ACCESS array[i]



PROCESS ELEMENT



NEXT INDEX



REPEAT

5 Things to Remember

1. Loops can process array elements
2. **i** is commonly used as the index
3. Index starts from 0
4. Loop continues until the condition becomes FALSE
5. Loops can input, display, and calculate array values

Final Example

```
int numbers[3] = {10, 20, 30};
```

```
for (int i = 0; i < 3; i++) {  
    printf("%d ", numbers[i]);  
}
```

Output:

10 20 30

⚡ Memory Trick

LOOP → VISIT EACH ELEMENT ↺

i → INDEX 📌

array[i] → CURRENT ELEMENT 🎯

QUESTIONS

1. Array Input Using a Loop

Complete the code to take input for all 5 array elements.

```
int numbers[5];
```

```
for (int i = 0; i < 5; i++) {  
    scanf("%d", &numbers[____]);  
}
```

💡 Hint: Each iteration should store the input at the current index.

Answer: **i**

2. What is the value of `numbers[2]`?

```
int numbers[5] = {10, 20, 30, 40, 50};
```

 **Hint:** Index starts from 0.

 **Glossary:** Index → Position of an element.

 **Answer:** 30

3. How many times does this loop run?

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]);  
}
```

 **Hint:** Values of `i` are 0, 1, 2, 3, and 4.

 **Glossary:** Iteration → One execution of a loop.

 **Answer:** 5 times

4. Arrange the steps in the correct order:

- A. Process the element
- B. Start the loop
- C. Access array[i]
- D. Move to the next index

 **Hint:** Start → Access → Process → Next.

 **Glossary:** Index → Position used to access an element.

 **Answer:** B → C → A → D

5. Print the Array in Reverse

Complete the loop condition and update to print the elements from last to first.

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
for (int i = 4; i >= 0; i____) {  
    printf("%d ", numbers[i]);  
}
```

💡 Hint: The index should decrease after every iteration.

Answer: --